

Capítulo 1

Preliminares

En este capítulo se presentan las definiciones y conceptos que serán usados en el resto del trabajo. Primero, en la sección 1.1, se introducen las nociones básicas de la teoría de lenguajes: alfabetos, cadenas, lenguajes, gramáticas y la jerarquía de Chomsky. Después, en la sección 1.2, se presenta el autómata finito. Por último, en las secciones 1.3 y 1.4, se introduce la complejidad computacional y se presenta el problema de la satisfacibilidad booleana con sus variantes polinomiales.

1.1. Teoría de Lenguajes

En esta sección se presentan los conceptos de la teoría de lenguajes que sirven de base al contenido de los capítulos posteriores. La sección se organiza en tres partes. Primero se introducen los conceptos de alfabeto, cadena y lenguaje, junto con los problemas de la palabra y del vacío. Después se definen las gramáticas, un mecanismo que permite describir lenguajes formales. Para finalizar se presenta la jerarquía de Chomsky, que clasifica los lenguajes formales según las restricciones en las reglas de producción de las gramáticas que los generan.

1.1.1. Conceptos básicos

Los conceptos básicos de la teoría de lenguajes formales son: alfabeto, cadena y lenguaje. Un **alfabeto**, denotado como Σ , es un conjunto finito y no vacío de símbolos. Una **cadena** es una sucesión finita de símbolos del alfabeto. Un **lenguaje** es un conjunto de cadenas definido sobre un alfabeto. Por ejemplo, el alfabeto $\Sigma = \{0, 1\}$ está formado por los símbolos 0 y 1; 11 y 101 son cadenas sobre el alfabeto Σ , y un ejemplo de lenguaje es el conjunto de cadenas formadas por 0 y 1 que terminan en 1.

El símbolo ε representa la **cadena vacía**. La **longitud** de una cadena w , denotada $|w|$, es la cantidad de símbolos que la componen; en particular, $|\varepsilon| = 0$.

Dadas dos cadenas w_1 y w_2 , la **concatenación** w_1w_2 se obtiene al escribir

los símbolos de w_1 seguidos de los símbolos de w_2 . Dada una cadena w y un entero $n \geq 0$, por convenio w^n representa la cadena resultante de concatenar w un total de n veces, con $w^0 = \varepsilon$.

Si Σ es un alfabeto, Σ^+ denota el conjunto de todas las cadenas de uno o más símbolos que se pueden formar sobre Σ , y Σ^* denota la unión de Σ^+ y $\{\varepsilon\}$.

Dado que los lenguajes son conjuntos, las operaciones sobre conjuntos también se definen para lenguajes: unión, intersección y complemento [6]. A continuación se formalizan dos problemas que se utilizan de manera recurrente en los capítulos posteriores: determinar si una cadena pertenece a un lenguaje y determinar si un lenguaje contiene algún elemento.

Definición 1.1. El **problema de la palabra** consiste en determinar si una cadena pertenece a un lenguaje dado.

Por ejemplo, dado el lenguaje L de las cadenas binarias que terminan en 0, el problema de la palabra para la cadena 1100100 consiste en determinar si $1100100 \in L$.

Definición 1.2. El **problema del vacío** consiste en determinar si un lenguaje es vacío.

Por ejemplo, el problema del vacío para el lenguaje formado por los números pares mayores que 5 que además son primos, consiste en decidir si tal lenguaje tiene algún elemento. Los antecedentes del presente trabajo, que se revisan en el capítulo ??, emplean el problema del vacío para decidir la satisfacibilidad de fórmulas booleanas, problema que se presenta en la sección 1.4.

Una vez introducidos los conceptos básicos y los dos problemas anteriores, a continuación se presentan las gramáticas, un mecanismo que permite describir los elementos de un lenguaje mediante reglas de producción.

1.1.2. Gramáticas

Los lenguajes formales se pueden describir mediante **gramáticas**, un mecanismo que permite generar los elementos de un lenguaje a partir de un símbolo inicial y un conjunto de reglas de producción.

Definición 1.3. Una **gramática** es un formalismo utilizado para describir lenguajes formales. Se define como una 4-tupla

$$G = (N, \Sigma, P, S),$$

donde

- N es un conjunto finito de símbolos llamados **símbolos no terminales**;
- Σ es un conjunto finito de símbolos llamados **símbolos terminales**, que constituyen el alfabeto sobre el que se construyen las cadenas del lenguaje. Se debe cumplir que $N \cap \Sigma = \emptyset$;

- P es un conjunto finito de **reglas de producción** de la forma

$$\alpha \rightarrow \beta, \quad \text{con } \alpha, \beta \in (N \cup \Sigma)^*;$$

- $S \in N$ es el **símbolo inicial** de la gramática.

Una **derivación** en la gramática consiste en seleccionar una regla de producción $\alpha \rightarrow \beta$ y sustituir una ocurrencia de α en una cadena $w \in (\Sigma \cup N)^+$ por β [6].

Por ejemplo, considere la gramática $G = (\{S\}, \{a, b\}, P, S)$ con producciones $P = \{S \rightarrow aSb, S \rightarrow ab\}$. Partiendo de S , una primera derivación aplica $S \rightarrow aSb$ y produce la cadena aSb ; una segunda derivación aplica $S \rightarrow ab$ sobre la ocurrencia de S en aSb y produce $aabb$.

Se dice que la gramática G genera una cadena $w \in \Sigma^*$ si existe una sucesión finita de derivaciones que, partiendo de S , produce w . El **lenguaje generado** por una gramática G se denota

$$L(G) = \{w \in \Sigma^* \mid S \xrightarrow{+} w\},$$

donde $\xrightarrow{+}$ indica la existencia de una sucesión finita de una o más derivaciones desde S hasta w . En el ejemplo anterior, $L(G) = \{a^n b^n \mid n \geq 1\}$.

Una vez presentadas las gramáticas, a continuación se expone la jerarquía de Chomsky, que clasifica los lenguajes formales según las restricciones en las reglas de producción de las gramáticas que los generan.

1.1.3. Jerarquía de Chomsky

La **Jerarquía de Chomsky** [7] clasifica los lenguajes en cuatro tipos, según las restricciones en las reglas de producción de las gramáticas que los generan.

El primer tipo son las **gramáticas irrestrictas**, que no imponen restricciones sobre sus reglas de producción. Cada regla tiene la forma $\alpha \rightarrow \beta$, donde $\alpha, \beta \in (N \cup \Sigma)^*$ y $\alpha \neq \varepsilon$. Todo lenguaje generado por una gramática irrestricta se denomina **lenguaje recursivamente enumerable** [REF].

El segundo tipo son las **gramáticas dependientes del contexto**. En estas, cada regla tiene la forma $\alpha A \gamma \rightarrow \alpha \beta \gamma$, donde $A \in N$, $\alpha, \beta, \gamma \in (N \cup \Sigma)^*$, y $|\beta| \geq 1$. Todo lenguaje generado por una gramática dependiente del contexto se denomina **lenguaje dependiente del contexto**. Todo lenguaje dependiente del contexto es también un lenguaje recursivamente enumerable. En [4] se demuestra que el lenguaje de las fórmulas booleanas satisfacibles no es libre del contexto, por lo que necesariamente se sitúa en alguno de estos dos tipos superiores; este resultado se discute en el capítulo ??.

El tercer tipo son las **gramáticas libres del contexto** (CFG). En estas, cada regla tiene la forma $A \rightarrow \beta$, donde $A \in N$ y $\beta \in (N \cup \Sigma)^*$. Todo lenguaje generado por una gramática libre del contexto se denomina **lenguaje libre del contexto**. Todo lenguaje libre del contexto es también un lenguaje dependiente del contexto [6].

Las gramáticas de concatenación de rango, formalismo central del capítulo ??, se introducen como una extensión de las CFG. Además, en [3] se emplea una CFG para resolver instancias del problema de la satisfacibilidad booleana, que se presenta en la sección 1.4.

El cuarto tipo son las **gramáticas regulares**. En estas, las reglas de producción tienen la forma

$$A \rightarrow aB \quad \text{o} \quad A \rightarrow a,$$

donde $A, B \in N$ y $a \in \Sigma$.

Todo lenguaje generado por una gramática regular se denomina **lenguaje regular**. Todo lenguaje regular es un lenguaje libre del contexto. Los lenguajes regulares se relacionan con los autómatas finitos, que se presentan en la sección 1.2.

Cada tipo de la jerarquía de Chomsky agrupa los lenguajes generados por las gramáticas de ese tipo. Estas agrupaciones se denominan **clases de lenguajes**: la clase de los lenguajes regulares, la de los libres del contexto, la de los dependientes del contexto y la de los recursivamente enumerables. Por brevedad, expresiones como "los lenguajes libres del contexto" se refieren a la clase correspondiente.

Una clase de lenguajes $\mathcal{A}$ es más expresiva que una clase $\mathcal{B}$ si todo lenguaje de $\mathcal{B}$ pertenece también a $\mathcal{A}$ y existe al menos un lenguaje de $\mathcal{A}$ que no pertenece a $\mathcal{B}$. Por ejemplo, la clase de los lenguajes libres del contexto es más expresiva que la clase de los lenguajes regulares [6].

La diferencia entre las clases de lenguajes de la jerarquía de Chomsky se puede ilustrar con una familia¹ de lenguajes derivada del lenguaje Copy. Dado un alfabeto Σ , para cada entero $k \geq 1$ se define el lenguaje

$$L_{\text{copy}}^k = \{w^k \mid w \in \Sigma^*\}.$$

La familia $\{L_{\text{copy}}^k\}_{k \geq 1}$ es la que ilustra la jerarquía. El caso $k = 1$ corresponde a $L_{\text{copy}}^1 = \Sigma^*$, que es un lenguaje regular. Para $k \geq 2$, el lenguaje L_{copy}^k no es libre del contexto, aunque sí es dependiente del contexto [6]. En el capítulo ?? se emplea este lenguaje para ilustrar el poder expresivo de las gramáticas de concatenación de rango.

La relación de inclusión entre las clases de la jerarquía se ilustra en la figura 1.1: una clase está contenida en otra cuando la elipse que la representa aparece dentro de la otra.

¹El término "familia de lenguajes" se utiliza aquí en un sentido distinto al de "clase de lenguajes" definido en el párrafo anterior: se refiere a una colección de lenguajes que se obtiene al variar un parámetro —en este caso, los lenguajes L_{copy}^k al variar k — y no a una clase de la jerarquía de Chomsky.

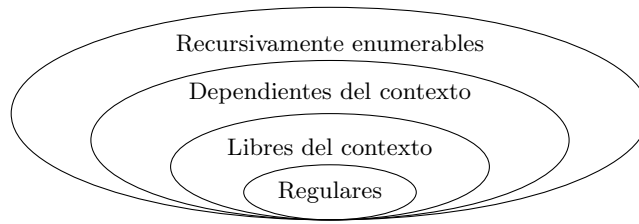


Figura 1.1: Jerarquía de Chomsky.

Cada clase de la jerarquía de Chomsky admite un modelo matemático, llamado autómata, que reconoce los lenguajes de la clase. En la sección 1.2 se define el autómata finito.

1.2. Autómatas finitos

Los autómatas correspondientes a las cuatro clases de la jerarquía de Chomsky son: la máquina de Turing para los lenguajes recursivamente enumerables, la máquina de Turing linealmente acotada para los lenguajes dependientes del contexto, el autómata de pila para los lenguajes libres del contexto y el autómata finito para los lenguajes regulares [6].

Definición 1.4. Un **autómata finito** [6] se define como una 5-tupla

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F),$$

donde Q es un conjunto finito de **estados**, Σ es un alfabeto llamado **alfabeto de entrada**, $\delta : Q \times \Sigma \rightarrow Q$ es una **función de transición**, $q_0 \in Q$ es el **estado inicial** y $F \subseteq Q$ es el conjunto de **estados de aceptación**.

El autómata comienza en el estado inicial q_0 y lee los símbolos de la cadena uno a uno. En cada paso, la función de transición δ determina, a partir del símbolo actual y del estado actual, el estado al que pasa el autómata. Al finalizar la lectura del último símbolo, si el autómata se encuentra en un estado de F , la cadena se acepta; en caso contrario, se rechaza.

Una vez presentado el autómata finito, en la próxima sección se introducen los conceptos de complejidad computacional, que se utilizan en los capítulos posteriores para analizar la complejidad del problema del vacío en los formalismos estudiados.

1.3. Complejidad computacional

Esta sección se organiza en tres partes. Primero se presenta la notación asintótica. Después se introduce la reducción polinomial. Por último se definen las clases de problemas P , NP y NP -Completo.

1.3.1. Notación asintótica

La notación asintótica permite describir el crecimiento de una función $f(n)$ a medida que n tiende a infinito. Se utiliza para clasificar la dificultad de un problema según el tiempo de ejecución de los algoritmos que lo resuelven.

Definición 1.5. Una función $g(n)$ pertenece a $O(f(n))$ si existen constantes positivas c y n_0 tales que

$$g(n) \leq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

La definición anterior expresa que $g(n)$ está acotada superiormente por un múltiplo constante de $f(n)$ para n suficientemente grande.

El tiempo de ejecución de un algoritmo es una función del tamaño n de su entrada. Se dice que un algoritmo tiene **tiempo polinomial** si esta función pertenece a $O(n^k)$ para alguna constante k . La **complejidad computacional** de un problema se define como el tiempo de ejecución, en el peor caso, del algoritmo más eficiente que lo resuelve [6].

Una vez introducida la noción de tiempo polinomial, en la próxima subsección se introduce la reducción polinomial como herramienta para relacionar la dificultad entre problemas.

1.3.2. Reducción polinomial

Los conceptos que se introducen a continuación, y los de la próxima subsección, se aplican a problemas de decisión. Un **problema de decisión** es un problema cuya respuesta para cada instancia es sí o no.²

Definición 1.6. Una **reducción polinomial** de un problema de decisión A a un problema de decisión B es una función f que se calcula en tiempo polinomial y asigna a cada entrada x de A una entrada $f(x)$ de B , tal que A responde sí sobre x si y solo si B responde sí sobre $f(x)$ [6].

La existencia de una reducción polinomial de A a B significa que B es, en términos de complejidad polinomial, al menos tan difícil como A . Como consecuencia, si B se resuelve en tiempo polinomial, A también. La reducción polinomial es la herramienta con la que se define la clase NP -Completo en la próxima subsección.

1.3.3. Clases de problemas

Los problemas computacionales [6] se agrupan en clases según el tiempo de ejecución necesario para resolverlos o verificar sus soluciones. En este trabajo se emplean las clases P , NP y NP -Completo.

Definición 1.7. Un problema pertenece a la clase P si se puede resolver en tiempo polinomial [6].

²Todo problema computacional admite una reformulación equivalente como problema de decisión [6]; trabajar con este tipo de problemas simplifica el análisis.

Por ejemplo, el problema 2-SAT, que se presenta en la sección 1.4.2, pertenece a P .

Definición 1.8. Un problema pertenece a la clase NP si existe un algoritmo que, dada una instancia del problema y una solución candidata, verifica en tiempo polinomial si la solución candidata es correcta [6].

Por ejemplo, el problema SAT, que se presenta en la sección 1.4, pertenece a NP : dada una asignación de valores a las variables de una fórmula booleana, un algoritmo polinomial determina si la asignación satisface la fórmula.

Definición 1.9. Un problema pertenece a la clase **NP -Completo** si pertenece a NP y, además, cualquier problema en NP se puede reducir polinomialmente a él [6].

El SAT del ejemplo anterior es también NP -Completo.

La pregunta de si $P = NP$ o $P \neq NP$ permanece sin resolver y constituye uno de los problemas abiertos centrales de la teoría de la computación [6]. El conjunto de problemas NP -Completo proporciona una base para el estudio de esta pregunta: si algún problema NP -Completo se puede resolver en tiempo polinomial, entonces todos los problemas de NP también.

Una vez introducidas estas clases, en la próxima sección se presenta el problema de la satisfacibilidad booleana, problema de decisión cuyo estudio mediante formalismos de la teoría de lenguajes motiva este trabajo.

1.4. Problema de la satisfacibilidad booleana

El problema de la satisfacibilidad booleana (SAT) es un problema de la teoría de la computación y la lógica matemática [6]. La sección se organiza en dos partes. Primero se presentan los elementos del SAT y se da su definición formal. Después se analiza el estatus del SAT como problema NP -Completo junto con sus variantes polinomiales.

1.4.1. Elementos del SAT

A continuación se presentan los elementos del SAT: variables booleanas, literales, cláusulas y fórmulas en forma normal conjuntiva.

- **Variables booleanas:** una variable booleana es una variable que puede tomar uno de dos valores: *verdadero* o *falso*.
- **Literales:** un literal es una variable booleana o su negación. Formalmente, si x es una variable booleana, entonces x y $\neg x$ son literales, donde $\neg x$ representa la negación de x . Un literal toma los valores *verdadero* o *falso* según la asignación de valores a las variables.

- **Cláusulas:** una cláusula es una disyunción, con el operador OR, de uno o más literales. Por ejemplo, la cláusula $(x \vee \neg y \vee z)$ es una disyunción de tres literales: x , $\neg y$ y z . Una cláusula es verdadera si al menos uno de sus literales es verdadero; es falsa si todos sus literales son falsos.
- **Fórmulas en forma normal conjuntiva:** una fórmula booleana en forma normal conjuntiva, abreviada CNF, es una conjunción con el operador AND de cláusulas. Por ejemplo,

$$(x \vee \neg y \vee z) \wedge (\neg x \vee y) \wedge (x \vee \neg z)$$

es una fórmula en CNF con tres cláusulas y tres variables.

Una **interpretación** de una fórmula booleana es una asignación de valores *verdadero* o *falso* a cada una de sus variables. Toda fórmula booleana se puede transformar, en tiempo polinomial, en una fórmula en CNF que es satisfacible si y solo si la fórmula original lo es [6]. Por esta razón, en el resto del trabajo se asume sin pérdida de generalidad que toda fórmula booleana está en CNF.

Definición 1.10. El problema de la **satisfacibilidad booleana**, o SAT, consiste en determinar si existe una asignación de valores *verdadero* o *falso* a las variables de una fórmula booleana, tal que la fórmula completa sea verdadera.

Una fórmula booleana en CNF es **satisfacible** si existe una asignación de valores a las variables tal que todas las cláusulas de la fórmula sean verdaderas simultáneamente.

Una vez definido el problema y sus elementos, a continuación se sitúa al SAT dentro de la clasificación de complejidad y se describen sus variantes polinomiales.

1.4.2. SAT como problema *NP*-Completo y variantes polinomiales

El SAT es el primer problema demostrado como *NP*-Completo y juega un rol central en la teoría de la complejidad computacional. El SAT pertenece a la clase *NP* porque, dada una asignación de valores a las variables de la fórmula, se puede verificar en tiempo polinomial si la asignación satisface la fórmula. La demostración de que el SAT es *NP*-Completo fue una de las contribuciones principales de Stephen Cook en 1971 [1], y marcó el inicio de la teoría de la *NP*-Complejidad.

Un SAT en el que cada cláusula tiene exactamente n literales distintos se denomina n -SAT. Para el problema 2-SAT existe una solución polinomial que determina si la fórmula booleana es satisfacible o no [5]. Para el problema 3-SAT no se conoce ningún algoritmo polinomial que permita determinar si una fórmula booleana es satisfacible o no [6]. Toda fórmula booleana del problema n -SAT se puede reducir a una fórmula equivalente del problema 3-SAT, por lo que SAT y 3-SAT son equivalentes en términos de complejidad computacional [6].

Aunque no se conoce algoritmo polinomial para resolver el SAT general, existen variantes particulares que sí admiten solución en tiempo polinomial, como 2-SAT, Horn-SAT y XOR-SAT.

El problema **2-SAT** es una variante del SAT donde cada cláusula contiene exactamente dos literales. El problema 2-SAT admite solución en tiempo polinomial mediante una modelación basada en grafos, con algoritmos como la detección de componentes fuertemente conexas en el grafo de implicación [5].

El problema **Horn-SAT** es una variante del SAT donde cada cláusula tiene a lo sumo un literal positivo. El problema Horn-SAT admite solución en tiempo polinomial mediante el algoritmo de resolución de Horn [2].

El problema **XOR-SAT** es una variante del SAT donde cada cláusula representa una operación XOR sobre sus literales. El problema XOR-SAT admite solución en tiempo polinomial al transformarlo en un sistema de ecuaciones lineales módulo 2 y aplicar eliminación de Gauss [8].

Este trabajo busca ampliar el conjunto de subclases del SAT con solución en tiempo polinomial mediante las gramáticas de concatenación de rango. En el próximo capítulo se presentan estas gramáticas, se analizan sus propiedades y limitaciones, y se revisan los antecedentes directos que abordan el SAT mediante formalismos de la teoría de lenguajes, con énfasis en los trabajos previos realizados en la Facultad de Matemática y Computación de la Universidad de La Habana.

Bibliografía

- [1] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, New York, NY, USA, 1971. ACM.
- [2] William F. Dowling and Jean H. Gallier. Linear-time algorithms for testing the satisfiability of propositional Horn formulae. *Journal of Logic Programming*, 1(3):267–284, 1984.
- [3] Alina Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. un algoritmo polinomial. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2007.
- [4] Raudel Alejandro Gómez Molina. Una aproximación al lenguaje de todas las fórmulas booleanas satisfacibles mediante gramáticas de concatenación de rango. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, February 2025.
- [5] Steven Halim, Felix Halim, and Suhendry Effendy. *Competitive Programming 4 – Book 2: The Lower Bound of Programming Contests in the 2020s*. Lulu.com, 4 edition, 2020.
- [6] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson Addison-Wesley, Boston, MA, 3 edition, 2006.
- [7] Tim Hunter. The Chomsky hierarchy. In Nicholas Allott, Terje Lohndal, and Georges Rey, editors, *A Companion to Chomsky*, chapter 5, pages 74–95. Wiley-Blackwell, 2021.
- [8] Thomas J. Schaefer. The complexity of satisfiability problems. In *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*, pages 216–226, New York, NY, USA, 1978. ACM.